

SeatForge Ticketing Platform

Software Engineering Portfolio

Qihui Pan | September 2026 | Personal project

[Public source code](#) / [Verified CI results](#)

SeatForge is a full-stack ticket reservation project focused on keeping inventory and orders consistent when buyers compete for the same seat or repeat checkout requests. It combines a Next.js buyer interface, a Spring Boot booking API, and a separate notification worker in a reproducible Docker environment.

The problem

A ticket sale can accept competing requests for a limited seat, lose progress when a hold expires, or duplicate a payment record after a retry. SeatForge makes these state transitions explicit and enforces the booking rules inside PostgreSQL transactions. The result is a working demonstration that reviewers can run locally and inspect through public tests.

What the project delivers

- **Buyer journey.** Browse events, view seat availability, hold a seat, create an order, simulate payment capture, and download a QR ticket image. Order history and refund APIs complete the lifecycle.
- **Catalog and access control.** Buyer registration and login, JWT authentication, ownership checks, and organizer APIs for event publishing and session creation.
- **Reliable background processing.** Transactional outbox publishing, persistent RabbitMQ messages, consumer deduplication, retries, and a dead-letter queue.
- **Operational visibility.** Prometheus metrics, Grafana configuration, and an optional OpenTelemetry integration linking request logs and metric exemplars to traces.

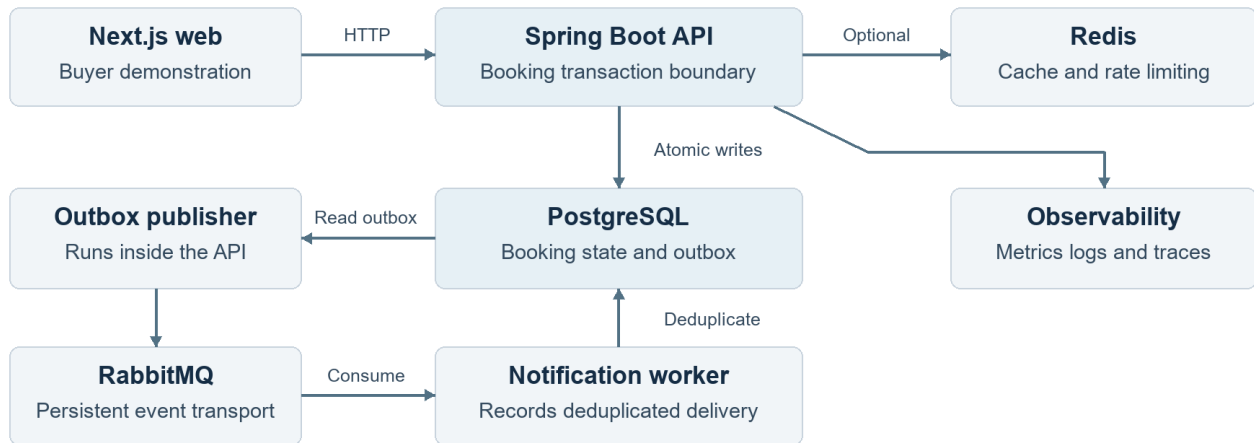
Technology and implementation scope

Layer	Implementation
Application	Java 21, Spring Boot 3.5, Next.js 16, React 19, TypeScript
Data and messaging	PostgreSQL 17, Flyway, Redis 8, RabbitMQ 4
Quality and delivery	JUnit, Testcontainers, k6, Docker Compose, GitHub Actions, Terraform

The portfolio covers backend transaction design, frontend integration, automated verification, observability, and deployment packaging. The source is public under the MIT License.

Architecture and engineering decisions

A modular monolith keeps the booking transaction in one place. The notification worker runs independently so message handling does not extend checkout latency.



PostgreSQL owns booking correctness. Redis accelerates reads; the outbox moves committed events to RabbitMQ.

Seat ownership under contention

A row lock serializes requests for the same seat. A partial unique index permits only one ACTIVE hold per seat, providing a second correctness check. Holds last five minutes; an expiry job claims batches with SKIP LOCKED so multiple API instances can release expired inventory without claiming the same work.

Checkout retries and atomic state changes

An idempotency key is serialized before the order is locked. The payment record, confirmed order, hold, sold seat, ticket code, audit entry, and outbox event are committed together. Repeating the same request returns the existing payment identifier. Refunds reopen inventory and support repeat requests.

Durable events with explicit deduplication

The API first stores the event in the booking transaction. A publisher sends it to RabbitMQ and waits for confirmation. Redelivery is possible, so the worker uses the event identifier as a unique delivery key. The current worker records delivery deterministically; an external email provider is a future integration.

Cache and observability boundaries

Redis failures fall back to PostgreSQL, and inventory changes invalidate cached availability after commit. Bounded HTTP metric labels avoid user identifiers and raw request paths. The optional OpenTelemetry overlay adds request traces, correlated logs, and Prometheus exemplars without requiring the external observability platform for local startup.

Tradeoff

Keeping the write path together simplifies consistency and deployment. A popular seat remains a serialized database resource, so horizontal API scaling still requires realistic database capacity planning and load measurement.

Verification and project handoff

The public CI run on September 7, 2026 passed for application revision d9577d3. The results below are reproducible test evidence, not production throughput or uptime measurements.

Check	Observed result	Evidence
Seat contention	100 simultaneous attempts; exactly 1 successful hold	PostgreSQL integration test
Payment retries	25 concurrent retries; 1 payment identifier and 1 outbox event	PostgreSQL integration test
Backend suite	10 tests passed; 0 failures, errors, or skipped tests	Maven CI log
Booking smoke test	19 of 19 checks passed; 0 failed HTTP requests out of 13	k6 fresh-install CI log
Build and configuration	Web verification, 3 container builds, and Terraform validation passed	GitHub Actions

Run the demonstration

Install Docker with Compose v2, then clone the repository and run the platform-specific helper. The helper creates local configuration, builds the images, and waits for API readiness.

```
git clone https://github.com/QihuiPan/ticketing-platform.git
cd ticketing-platform
```

Windows: powershell -ExecutionPolicy Bypass -File .\scripts\quickstart.ps1

macOS / Linux: ./scripts/quickstart.sh

Open <http://localhost:3000>. The README provides local demo accounts. Run the documented k6 smoke test to verify checkout, payment replay, QR download, refund, and restored availability.

Current delivery scope

The project provides a runnable demonstration with simulated payment capture and recorded notification delivery. AWS Terraform includes a single-node demo and a production infrastructure baseline. Public application hosting and a live deployment URL are not part of this handoff. Real payment and email providers, environment-specific HTTPS and secrets, and production load validation remain deployment work.

Review the implementation

[Test results](#) | [Concurrency tests](#) | [Architecture](#) | [Run guide](#)

Portfolio prepared September 11, 2026. Application evidence is pinned to d9577d3; later documentation commits do not change that tested application revision.